Universidad Nacional Autónoma de México
Facultad de Ingeniería
Fundamentos de Sistemas Embebidos
Tarea 3: Nombres con sentido
Rafael Ortega de la Paz | 420054085
09/09/26

En el segundo capítulo de Código Limpio, Tim Ottinger explica que, dado que los nombres aparecen constantemente en el software —en variables, funciones, clases, archivos—, elegirlos bien no es un detalle menor, sino una práctica que ahorra mucho trabajo a quien después lea el código. Un buen nombre debe revelar por sí solo por qué existe algo, qué hace y cómo se usa: si a una variable o función se le da un nombre con sentido, cualquier programador puede obtener contexto de ella de inmediato, entender para qué se utiliza y cuál es el propósito del código sin necesidad de comentarios adicionales.

El autor advierte que los nombres también pueden desinformar: usar palabras cuyo significado se aleje de lo que realmente representan, o crear variaciones mínimas entre nombres parecidos, confunde más de lo que ayuda. Del mismo modo, distinguir nombres solo para satisfacer al compilador —añadiendo números o palabras huecas como Info o Data— no aporta ningún significado real; la distinción debe ser semántica, no cosmética.

Una idea central del capítulo es que no debemos codificar más de lo necesario. Ottinger critica prácticas como la notación húngara o los prefijos de tipo (m_, I, etc.), heredadas de una época en la que los compiladores no verificaban tipos. En los lenguajes modernos, esas codificaciones ya no aportan nada: solo añaden una carga mental innecesaria y dificultan cambiar el nombre o el tipo de una variable. Los nombres, además, deben poder pronunciarse y buscarse fácilmente, porque programar es una actividad social y el código se lee muchas más veces de las que se escribe.

Finalmente, se distingue el buen código del malo en la disciplina de sus nombres: las clases deben nombrarse con sustantivos, los métodos con verbos, evitando el exceso de ingenio o los juegos de palabras que solo el autor entiende. Usar una sola palabra por concepto y añadir contexto con sentido hace que el código sea comprensible por sí mismo, sin exigir al lector que investigue su significado.